

On-Policy vs Off-Policy Reinforcement Learning

Understanding Which Policy Generated the Data

1 Goal of This Note

The distinction between on-policy and off-policy learning is one of the most important ideas in reinforcement learning.

The key question is:

Was the training data generated by the same policy we are currently improving?

If yes, the method is on-policy. If no, the method is off-policy.

2 Two Policies

It helps to name two policies:

- Behavior policy: the policy that collects data.
- Target policy: the policy we want to learn or improve.

We can write:

$\mu(a \mid s)$ = behavior policy,

$\pi(a \mid s)$ = target policy.

The distinction is:

on-policy: $\mu = \pi$

off-policy: $\mu \neq \pi$

3 On-Policy Learning

An on-policy algorithm learns from data generated by its current policy.

The loop is:

$\pi_\theta \rightarrow$ collect fresh episodes $\rightarrow$ update $\pi_\theta \rightarrow$ discard old data

The data is fresh because it came from the policy before the update.

4 On-Policy Example: REINFORCE

REINFORCE samples actions from the current policy:

$$a_t \sim \pi_\theta(\cdot \mid s_t).$$

Then it updates the same policy using:

$$\mathcal{L}(\theta) = - \sum_t \log \pi_\theta(a_t \mid s_t) G_t.$$

This is on-policy because the sampled actions came from the policy being updated.

```
# on-policy sketch
for iteration in range(num_iterations):
    trajectories = collect_data(policy) # current policy
    loss = reinforce_loss(policy, trajectories)
    update(policy, loss)
    # old trajectories are usually discarded
```

5 Why On-Policy Data Becomes Stale

Suppose the old policy was:

$$\pi_{\text{old}}(\text{left} \mid s) = 0.9.$$

After an update, the new policy may be:

$$\pi_{\text{new}}(\text{left} \mid s) = 0.2.$$

Old data contains many left actions because the old policy preferred left. But the new policy no longer behaves that way.

So old data no longer represents the current policy well. On-policy methods usually need new data after each update.

6 Off-Policy Learning

An off-policy algorithm can learn from data generated by another policy.

The loop can be:

$$\mu \rightarrow \text{collect data} \rightarrow \text{store data} \rightarrow \text{train target policy } \pi$$

The behavior policy may be old, exploratory, random, human-generated, or another model.

7 Off-Policy Example: Q-Learning

Q-learning often collects data with epsilon-greedy exploration:

$$\mu = \epsilon\text{-greedy}(Q).$$

But its target uses the greedy action:

$$\max_{a'} Q(s', a').$$

So the behavior may be exploratory, but the learned target is greedy. This is off-policy.

```
# off-policy Q-learning sketch
action = epsilon_greedy_action(Q, state) # behavior policy
next_state, reward, done, info = env.step(action)

target = reward + gamma * max(Q[next_state]) # greedy target policy
Q[state][action] += alpha * (target - Q[state][action])
```

8 Replay Buffers

Replay buffers are naturally off-policy. They store old experience:

$$(s, a, r, s', d).$$

Later, the algorithm trains on that experience even though the current policy may be different from the policy that generated it.

```
replay_buffer.append((state, action, reward, next_state, done))

batch = random.sample(replay_buffer, batch_size)
loss = q_learning_loss(batch)
```

This is one reason off-policy methods can be more sample efficient. They reuse data.

9 Sample Efficiency

Sample efficiency means:

how much learning happens per environment interaction.

On-policy methods usually have lower sample efficiency because they discard old data. Off-policy methods usually have higher sample efficiency because they reuse old data.

But there is a tradeoff:

data reuse improves efficiency but can make learning less stable.

10 Distribution Mismatch

Off-policy learning must deal with distribution mismatch.

The data comes from:

$$\mu(a \mid s),$$

but we want to learn about:

$$\pi(a \mid s).$$

If μ almost never takes an action that π wants to take, the data may not tell us much about that action.

Example: if the behavior policy never tries action a , then we have no evidence about $Q(s, a)$.

11 Importance Sampling

One mathematical correction for off-policy data is importance sampling. If data comes from μ but we want expectation under π , we can weight samples by:

$$\rho_t = \frac{\pi(a_t | s_t)}{\mu(a_t | s_t)}.$$

If the target policy assigns high probability to an action and the behavior policy assigned low probability, the sample receives more weight.

$$\mathbb{E}_{a \sim \pi}[f(a)] = \mathbb{E}_{a \sim \mu} \left[\frac{\pi(a)}{\mu(a)} f(a) \right].$$

In practice, importance sampling can have high variance. Large ratios can make learning unstable.

12 Small Importance Sampling Example

```
target_prob = 0.80 # pi(a / s)
behavior_prob = 0.20 # mu(a / s)

rho = target_prob / behavior_prob
weighted_return = rho * observed_return
```

Here $\rho = 4$. The sample is weighted heavily because the target policy likes this action more than the behavior policy did.

13 Modern Examples

Algorithm	Type	Reason
REINFORCE	On-policy	Learns from current policy rollouts
A2C/A3C	On-policy	Updates from current actor data
PPO	Mostly on-policy	Uses recent rollouts with limited reuse
SARSA	On-policy	Target uses action actually taken
Q-learning	Off-policy	Target uses greedy action
DQN	Off-policy	Uses replay buffer and greedy target
DDPG/TD3/SAC	Off-policy	Reuse replay buffer data

14 SARSA vs Q-Learning

SARSA is useful for understanding the distinction.

SARSA update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)] .$$

Here a' is the action actually selected by the current behavior policy. So SARSA learns the value of the policy it follows. It is on-policy.

Q-learning update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] .$$

Q-learning uses the best possible next action in the target. It learns a greedy target policy even while behaving exploratorily. It is off-policy.

15 Code Comparison

```
# SARSA: on-policy target
next_action = epsilon_greedy_action(Q, next_state)
target = reward + gamma * Q[next_state][next_action]

# Q-learning: off-policy greedy target
target = reward + gamma * max(Q[next_state])
```

The only difference is the target. But that difference changes the learning type.

16 When to Use On-Policy Methods

On-policy methods are attractive when:

- the policy is stochastic and we want stable policy updates;
- the environment is cheap to sample;
- we want a direct policy optimization method;
- the action space is continuous or very large;
- we care about the behavior of the exact current policy.

PPO is popular because it is an on-policy style method with practical stability tricks.

17 When to Use Off-Policy Methods

Off-policy methods are attractive when:

- environment interaction is expensive;
- we want to reuse data many times;
- we can store experience in a replay buffer;
- we have offline datasets;

- we want to learn from demonstrations or older policies.

Offline RL is an extreme version of off-policy learning: the agent trains from a fixed dataset without collecting new environment data.

18 Connection to Language Models

For language models, the policy is:

$$\pi_{\theta}(\text{next token} \mid \text{prompt and previous tokens}).$$

On-policy RL would sample completions from the current model and update from those samples. Off-policy RL can train from completions generated by older models, humans, or stored datasets. The same question applies:

Did the current model generate the data, or did something else generate it?

19 Minimal Mental Model

The shortest way to remember the distinction is:

on-policy learns from what it currently does.

off-policy can learn from what something else did.

On-policy methods are usually cleaner and more direct. Off-policy methods are usually more data efficient but require more care.